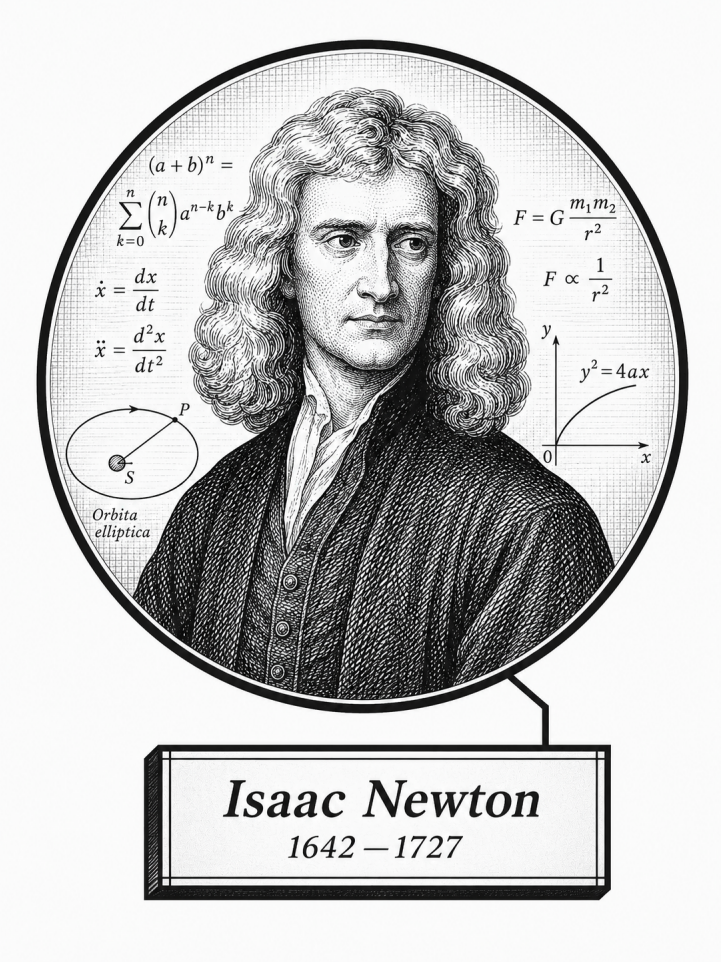
Contents

Volume VII

Numerical and Approximation Mathematics



Frontispiece placeholder.
Isaac Newton, 1642–1727

Volume VII

Numerical and Approximation Mathematics

Self-Study Notes

William Sollers

June 11, 2026

Part I

Numerical and Approximation Mathematics

Status: Planned

This volume is a structural stub created during the eight-volume migration. Content migration will occur in later phases.

Chapter 1

Numerical Analysis

1.0.1 IEEE 754 Floating-Point Formats

Toolkit — Floating-Point Representation

This section develops the structural language of IEEE 754 binary floating-point representation. The key ideas are:

- every binary floating-point datum is organized into sign, exponent, and fraction fields,
- the exponent field determines whether the encoding is normalized, subnormal, zero, infinity, or NaN,
- the fraction field determines the significant bits of the represented value,
- each standard format is determined by its total bit width together with its exponent width, fraction width, and exponent bias.

Breadcrumb

Volume IV → Computational Mathematics → Floating-Point Arithmetic → IEEE 754 Formats

Remark (Structural roadmap). We begin with the general structure of an IEEE 754 binary floating-point datum. We then define the principal interpretation classes: normalized numbers, subnormal numbers, infinities, and NaNs. After that, we define the standard binary interchange formats binary16, binary32, binary64, and binary128, and display the bit layout of each format using a common TikZ layout macro.

Definition — IEEE 754 Binary Floating-Point Datum

Definition 1.0.1 (IEEE 754 Binary Floating-Point Datum). An *IEEE 754 binary floating-point datum* is a binary encoding partitioned into three fields:

- a *sign field* consisting of one bit,
- an *exponent field* consisting of a fixed number of bits,
- a *fraction field* consisting of a fixed number of bits.

Its interpretation depends on the exponent-field pattern and the fraction-field pattern.

Remark (Interpretation). IEEE 754 does not interpret a floating-point bit string as a plain binary integer. Instead, the sign field determines sign, the exponent field determines scale and special cases, and the fraction field records the significant bits of the encoded value.

Remark (Definition predicate reading).

$$\mathrm{IEEEBinaryFloatDatum}(x) \iff \mathrm{HasSignField}(x) \wedge \mathrm{HasExponentField}(x) \wedge \mathrm{HasFractionField}(x).$$

Definition — Normalized IEEE 754 Binary Number

Definition 1.0.2 (Normalized IEEE 754 Binary Number). Let an IEEE 754 binary floating-point format have exponent bias bias. A finite encoded datum is called *normalized* if its exponent field is neither all zeros nor all ones.
If s is the sign bit, if e is the unsigned integer determined by the exponent field, and if f is the binary fraction determined by the fraction field, then the represented value is

$$(-1)^s(1.f)_2 2^{e-\text{bias}}.$$

Remark (Interpretation). In a normalized encoding, the leading binary digit of the significand is implicitly 1. This hidden leading bit is what gives normalized numbers one more bit of effective precision than the stored fraction field alone.

Remark (Definition predicate reading).

$\text{NormalizedIEEEBinaryNumber}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{ExponentFieldNeitherAllZerosNo}$

Definition — Subnormal IEEE 754 Binary Number

Definition 1.0.3 (Subnormal IEEE 754 Binary Number). Let an IEEE 754 binary floating-point format have exponent bias bias. A finite encoded datum is called *subnormal* if its exponent field is all zeros and its fraction field is not all zeros.
If s is the sign bit and if f is the binary fraction determined by the fraction field, then the represented value is

$$(-1)^s(0.f)_2 2^{1-\text{bias}}.$$

Remark (Interpretation). Subnormal numbers fill the gap between zero and the smallest positive normalized number. They preserve gradual underflow by allowing the leading significand bit to be 0 instead of the implicit 1 used for normalized numbers.

Remark (Definition predicate reading).

$\text{SubnormalIEEEBinaryNumber}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{ExponentFieldAllZeros}(x) \wedge \neg \text{Frac}$

Definition — IEEE 754 Infinity Encoding

Definition 1.0.4 (IEEE 754 Infinity Encoding). An IEEE 754 binary floating-point datum encodes *infinity* if its exponent field is all ones and its fraction field is all zeros.
The sign bit determines whether the represented value is $+\infty$ or $-\infty$.

Remark (Interpretation). Infinity is a distinguished non-finite encoding used to represent overflow and certain limit-like arithmetic results. The sign field still matters, so IEEE 754 distinguishes positive infinity from negative infinity.

Remark (Definition predicate reading).

$\mathrm{IEEEInfinityEncoding}(x) \iff \mathrm{IEEEBinaryFloatDatum}(x) \wedge \mathrm{ExponentFieldAllOnes}(x) \wedge \mathrm{FractionFieldA}$

Definition — IEEE 754 NaN Encoding

Definition 1.0.5 (IEEE 754 NaN Encoding). An IEEE 754 binary floating-point datum encodes *NaN* (Not a Number) if its exponent field is all ones and its fraction field is not all zeros.

Remark (Interpretation). A NaN encoding represents an undefined or invalid numerical result rather than a real number. For example, NaNs arise from operations such as invalid indeterminate forms or from the propagation of an already invalid datum through later computations.

Remark (Definition predicate reading).

$\mathrm{IEEENaNEncoding}(x) \iff \mathrm{IEEEBinaryFloatDatum}(x) \wedge \mathrm{ExponentFieldAllOnes}(x) \wedge \neg\, \mathrm{FractionFieldAl}$

Definition — IEEE 754 Binary16 Format

Definition 1.0.6 (IEEE 754 Binary16 Format). The *IEEE 754 binary16 format* is the 16-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 5 exponent bits,
- 10 fraction bits,
- exponent bias 15.

For a normalized finite binary16 datum, the represented value is

$$(-1)^s (1.f)_2 2^{e-15}.$$

Remark (Interpretation). Binary16 is commonly called *half precision*. It is useful when reduced storage and bandwidth matter more than wide dynamic range or high precision.

Remark (Definition predicate reading).

$\mathrm{Binary16}(x) \iff \mathrm{IEEEBinaryFloatDatum}(x) \wedge \mathrm{BitWidth}(x, 16) \wedge \mathrm{SignWidth}(x, 1) \wedge \mathrm{ExponentWidth}(x, 5$

Definition — IEEE 754 Binary32 Format

Definition 1.0.7 (IEEE 754 Binary32 Format). The *IEEE 754 binary32 format* is the 32-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 8 exponent bits,
- 23 fraction bits,
- exponent bias 127.

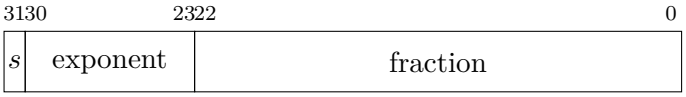
For a normalized finite binary32 datum, the represented value is

$$(-1)^s(1.f)_2 2^{e-127}.$$

Remark (Interpretation). Binary32 is commonly called *single precision*. It is the standard 32-bit floating-point format used broadly in scientific computing, computer graphics, simulation, and machine learning.

Remark (Definition predicate reading).

$\mathrm{Binary32}(x) \iff \mathrm{IEEEBinaryFloatDatum}(x) \wedge \mathrm{BitWidth}(x, 32) \wedge \mathrm{SignWidth}(x, 1) \wedge \mathrm{ExponentWidth}(x, 8$



Definition — IEEE 754 Binary64 Format

Definition 1.0.8 (IEEE 754 Binary64 Format). The *IEEE 754 binary64 format* is the 64-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 11 exponent bits,
- 52 fraction bits,
- exponent bias 1023.

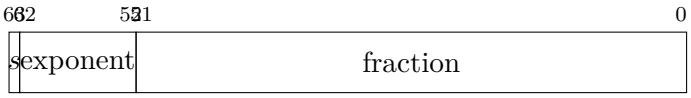
For a normalized finite binary64 datum, the represented value is

$$(-1)^s(1.f)_2 2^{e-1023}.$$

Remark (Interpretation). Binary64 is commonly called *double precision*. It is the standard high-precision floating-point format for much of scientific and engineering computation.

Remark (Definition predicate reading).

$\mathrm{Binary64}(x) \iff \mathrm{IEEEBinaryFloatDatum}(x) \wedge \mathrm{BitWidth}(x, 64) \wedge \mathrm{SignWidth}(x, 1) \wedge \mathrm{ExponentWidth}(x, 1$



Definition — IEEE 754 Binary128 Format

Definition 1.0.9 (IEEE 754 Binary128 Format). The *IEEE 754 binary128 format* is the 128-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 15 exponent bits,
- 112 fraction bits,
- exponent bias 16383.

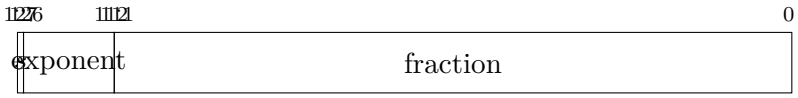
For a normalized finite binary128 datum, the represented value is

$$(-1)^s (1.f)_2 2^{e-16383}.$$

Remark (Interpretation). Binary128 is commonly called *quadruple precision*. It provides substantially greater range and precision than binary64 at substantially greater storage and computational cost.

Remark (Definition predicate reading).

$$\text{Binary128}(x) \iff \text{IEEEBinaryFloatDatum}(x) \wedge \text{BitWidth}(x, 128) \wedge \text{SignWidth}(x, 1) \wedge \text{ExponentWidth}(x$$



Remark (Summary). The four standard IEEE 754 basic binary interchange formats considered here are:

binary16 $(1, 5, 10)$, binary32 $(1, 8, 23)$, binary64 $(1, 11, 52)$, binary128 $(1, 15, 112)$,

where each triple records

$$(\text{sign bits},\ \text{exponent bits},\ \text{fraction bits}).$$

1.1 Interval Arithmetic

Interval Arithmetic - Quick Reference

Operation	Rule	Detail
Addition	$[a,b] + [c,d] = [a+c,\ b+d]$	Down to Def
Subtraction	$[a,b] - [c,d] = [a-d,\ b-c]$	Down to Def
Multiplication	$[a,b] \cdot [c,d]$ (case analysis)	Down to Def
Containment	$[a,b] \subseteq [c,d] \iff c \le a, b \le d$	Down to Def

Definition (Interval Addition)

For closed intervals $[a, b]$ and $[c, d]$, their *sum* is

$$[a, b] + [c, d] \;=\; [a + c,\; b + d].$$

Definition (Interval Subtraction)

For closed intervals $[a, b]$ and $[c, d]$, their *difference* is

$$[a, b] - [c, d] \;=\; [a - d,\; b - c].$$

Definition (Interval Multiplication)

For closed intervals $[a, b]$ and $[c, d]$, their *product* is

$$[a, b] \cdot [c, d] \;=\; \big[\min(ac, ad, bc, bd),\; \max(ac, ad, bc, bd)\big].$$

Definition (Interval Containment)

$[a, b] \subseteq [c, d]$ if and only if $c \le a$ and $b \le d$.

Remark 1.1.1 (Motivation). Interval arithmetic provides a framework for rigorous numerical computation with guaranteed error bounds. A real number $x \in [a, b]$ means the computed value may lie anywhere in the interval; operations propagate these bounds.

Remark 1.1.2 (Status). *This section is a stub. A full treatment - the sub-distributivity law, dependency problem, applications to root-finding, and connections to compactness - will be developed in a future revision.*

1.2 Interval Arithmetic in $\mathbb{R}^n$

Box (Interval) Arithmetic in $\mathbb{R}^n$ - Quick Reference

Operation	Rule	Detail
Box (product interval)	$[a, b] := \{x \in \mathbb{R}^n : a \le x \le b\}$	Down to Def
Containment	$[a, b] \subseteq [c, d] \iff c \le a \;\wedge\; b \le d$	Down to Def
Minkowski sum	$[a, b] \oplus [c, d] = [a + c,\; b + d]$	Down to Def
Difference	$[a, b] \ominus [c, d] = [a - d,\; b - c]$	Down to Def
Scalar mult. ($\lambda \ge 0$)	$\lambda[a, b] = [\lambda a,\; \lambda b]$	Down to Def
Scalar mult. ($\lambda < 0$)	$\lambda[a, b] = [\lambda b,\; \lambda a]$	Down to Def
Coordinatewise product	$[a, b] \odot [c, d] = \prod_i [\min S_i, \max S_i]$	Down to Def
Norm bound (Euclidean)	$\|x\|_2 \le \max\{\|a\|_2, \|b\|_2\}$ for $x \in [a, b]$ if $0 \le a \le b$	Down to Rem

Definition (Box / Product Interval)

Let $a, b \in \mathbb{R}^n$ with $a \leq b$ coordinatewise (i.e., $a_i \leq b_i$ for all i). The *box* (or *interval vector*) determined by (a, b) is

$$[a, b] \;:=\; \{x \in \mathbb{R}^n : a \leq x \leq b\} \;=\; \prod_{i=1}^{n} [a_i, b_i].$$

Definition (Box Containment)

For $a \leq b$ and $c \leq d$ in $\mathbb{R}^n$, we write $[a, b] \subseteq [c, d]$ iff

$$(\forall x \in \mathbb{R}^n)\ \big(x \in [a, b] \Rightarrow x \in [c, d]\big).$$

Equivalently (and most useful computationally),

$$[a, b] \subseteq [c, d] \quad \Longleftrightarrow \quad c \leq a \ \wedge\ b \leq d \qquad \text{(coordinatewise)}.$$

Definition (Minkowski Addition of Boxes)

For boxes $X = [a, b]$ and $Y = [c, d]$ in $\mathbb{R}^n$, their *Minkowski sum* is

$$X \oplus Y \;:=\; \{x + y : x \in X,\ y \in Y\}.$$

For boxes, this closes in the class of boxes and satisfies the endpoint rule

$$[a, b] \oplus [c, d] \;=\; [a + c,\ b + d].$$

Definition (Box Subtraction / Difference)

For boxes $X = [a, b]$ and $Y = [c, d]$ in $\mathbb{R}^n$, define

$$X \ominus Y \;:=\; \{x - y : x \in X,\ y \in Y\}.$$

Then

$$[a, b] \ominus [c, d] \;=\; [a - d,\ b - c].$$

Definition (Scalar Multiplication of a Box)

For $\lambda \in \mathbb{R}$ and a box $[a, b] \subseteq \mathbb{R}^n$, define

$$\lambda[a, b] \;:=\; \{\lambda x : x \in [a, b]\}.$$

Then

$$\lambda[a, b] = \begin{cases} [\lambda a,\ \lambda b], & \lambda \geq 0, \\ [\lambda b,\ \lambda a], & \lambda < 0, \end{cases}$$

where the inequalities are coordinatewise.

Definition (Coordinatewise / Hadamard Product of Boxes)

For $x, y \in \mathbb{R}^n$, write $(x \odot y)_i := x_i y_i$. For boxes $X = [a, b]$ and $Y = [c, d]$, define

$$X \odot Y \;:=\; \{x \odot y : x \in X,\; y \in Y\}.$$

Then $X \odot Y$ is again a box, computed coordinatewise:

$$[a, b] \odot [c, d] \;=\; \prod_{i=1}^{n} \Big[\min S_i,\; \max S_i\Big], \qquad S_i := \{a_i c_i,\; a_i d_i,\; b_i c_i,\; b_i d_i\}.$$

Remark 1.2.1 (Interpretation: intervals as guaranteed enclosures). A box $X = [a, b] \subseteq \mathbb{R}^n$ represents a *set of possible vectors* (e.g., a computed vector plus uncertainty bounds). The operations above are *set operations* (Minkowski sum/difference, scalar image), so the result is a *guaranteed enclosure* of all possible true outcomes under the corresponding vector operation.

Remark 1.2.2 (Norm bounds and why boxes are convenient). For many applications you also want quick bounds on a norm over a box. For example, if $0 \le a \le b$ coordinatewise, then for every $x \in [a, b]$ we have $0 \le x \le b$ and hence $\|x\|_2 \le \|b\|_2$. More generally, bounding norms over a box can be reduced to checking finitely many "corners" when the objective is coordinatewise monotone.

Remark 1.2.3 (Status). *This section is a stub. A full treatment - interval extensions of general functions $f : \mathbb{R}^n \to \mathbb{R}^m$, the dependency problem, sub-distributivity, natural vs. centered (affine) forms, and connections to compactness and continuity - will be developed in a future revision.*

1.3 Numerical Geometry

Status: Planned

This planned topic, *Points, Vectors, Error, and Interpolation*, will connect numerical error, point/vector geometry, interpolation, plotting, and the `lra-numerical-analysis` programming workspace.

1.3.1 Week 1: Points, Vectors, and Floating-Point Geometry

Reading checklist.

- Atkinson, Chapter 1
- Solomon, Chapter 1 selectively
- Solomon, Chapter 2 selectively
- Vince, Chapter 7
- Existing Volume VII IEEE 754 notes
- Existing Volume VII interval arithmetic notes

Project placeholder. `lra-numerical-analysis/projects/point-vector-01`

Handwritten notes placeholder. TODO: Add handwritten-note extraction here after study.

1.3.2 Week 2: Linear Interpolation and Affine Sampling

Reading checklist.

- Salomon, Chapter 1 sections 1.1–1.3
- Salomon, Chapter 2 section 2.1
- Vince, Chapter 13
- Piegl–Tiller, Chapter 1 skim only
- Piegl–Tiller, Chapter 2 preview only

Project placeholder. `lra-numerical-analysis/projects/linear-interpolation-02`

Handwritten notes placeholder. TODO: Add handwritten-note extraction here after study.

1.3.3 Projects

- `point-vector-01`: planned
- `linear-interpolation-02`: planned
- `bezier-curve-03`: deferred
- `bspline-basis-04`: deferred
- `nurbs-curve-05`: deferred

1.3.4 Exercises

- Week 1 exercises: planned
- Week 2 exercises: planned
- TODO: Add exercises after handwritten notes are extracted.

Proofs

Exercises

Remark (Capstone Exercise - Numerical Analysis). TODO: Capstone problem statement goes here.

The capstone must synthesize the core results of this chapter. It may reference any concept from this chapter or from prior chapters in the dependency order. It must not reference concepts from chapters that succeed this chapter in the registry.

Proof. TODO □

Chapter 2

Numerical PDE

Status: Planned

Active mathematical content has not yet been added.

Numerical PDE Notes

This chapter is planned. Active mathematical content has not yet been added.

Proofs

Proof entries for this chapter are planned. No proof files have been regenerated.

Exercises

Exercises for this chapter are planned. No exercise content has been restored here yet.

Bibliography